

Kafka Fraud Sentinel

Real-Time Fraud Detection System

Giacomo Comitani - 85673A

August 2026

Project Report – Cloud Computing Technologies
University of Milan (UNIMI), Academic Year 2025/2026

Abstract

This report presents Kafka Fraud Sentinel, a distributed, real-time fraud detection system built on Apache Kafka. Developed for the Cloud Computing Technologies course, the project simulates a Security Operations Center (SOC) capable of ingesting high volumes of financial transactions and instantly identifying anomalous patterns, such as velocity and amount frauds. The architecture is entirely containerized using Docker and leverages a 3-node KRaft cluster to ensure high availability and fault tolerance. Strict security boundaries are enforced through mutual TLS (mTLS) authentication and fine-grained Access Control Lists (ACLs). Verified alerts are persisted in MongoDB, streamed to a live web dashboard via WebSockets, and forwarded to administrators via a Telegram bot.

1 Project Overview

For this Cloud Computing project, a distributed system was developed that analyzes financial transactions in real time to catch potential frauds.

Instead of building a classic REST API where services talk directly to each other, an event-driven approach based on Apache Kafka was chosen. In a real-world bank, thousands of transactions happen every second. If the fraud detection engine experiences a temporary outage, the system cannot afford to lose the transactions or block users from making payments. Kafka acts as a durable buffer: producers simply send transactions to a topic, and consumers process them at their own pace.

The main goals of the project are:

- Real-time data processing using Python.
- A secure environment using SSL certificates (mTLS) and ACLs, ensuring no unauthorized access.
- High availability, ensuring the system survives if a server crashes.

2 Infrastructure Setup

The entire infrastructure is containerized using Docker Compose. The core is a Kafka cluster composed of 3 brokers running in KRaft mode, meaning there is no ZooKeeper dependency. A MongoDB container is also included to save the alerts, along with AKHQ to monitor the cluster via a web interface.

2.1 Topics and Partitioning

During the Kafka setup, two separate topics were created. This separation is a key design choice:

- **transactions**: this is the main topic where all the generated transactions arrive. It handles high traffic.
- **fraud-alerts**: this topic only receives the suspicious transactions. It has very low traffic.

Both topics are configured with 3 partitions and a replication factor of 3. The `min.insync.replicas` parameter is set to 2. This setup guarantees that every message is copied on all three brokers, and a write is considered successful only if at least two brokers save it on disk.

When producing messages to the **transactions** topic, the `user_id` is used as the Kafka key. This is fundamental for the fraud detection logic. Kafka guarantees that messages with the same key always go to the same partition. This means the consumer reads the transactions of a specific user in the exact chronological order they were made, which is mandatory to detect velocity frauds accurately.

3 Application Components

The codebase is divided into distinct Python microservices. Figure 1 shows how they interact.

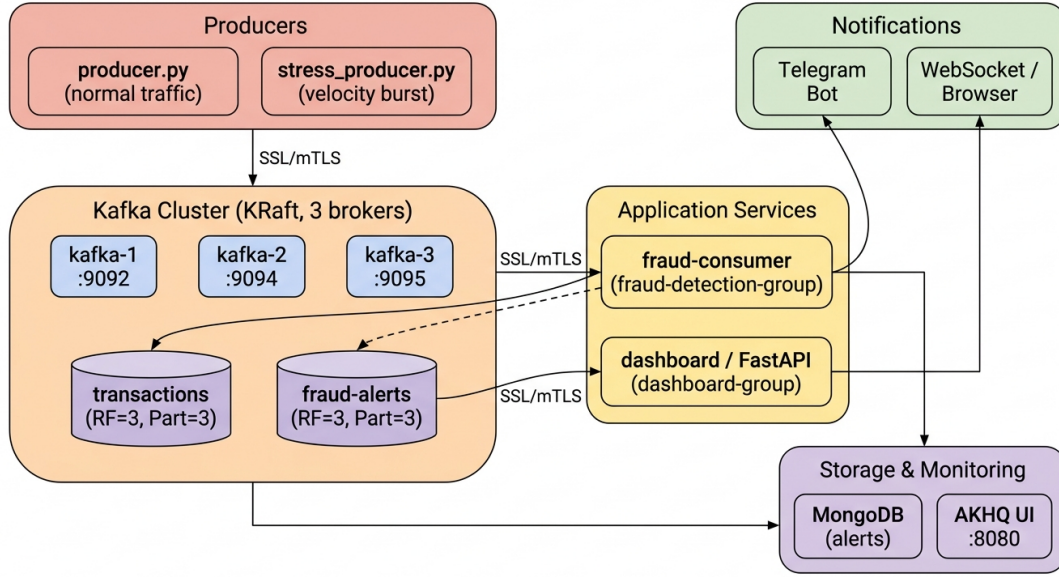


Figure 1: Architecture of the system.

3.1 Data Generation (Producers)

To test the system, a `producer.py` script continuously generates mock transactions for 100 different users. It is programmed to mix normal traffic with fraudulent behavior:

- About 30% of the transactions have a high amount (between 1000 and 8500 EUR). Those exceeding 5000 EUR are flagged as Amount Fraud by the consumer.
- About 15% of the time, the producer picks a random user and sends a "burst" of 4 to 6 transactions in less than a second.

A `stress_producer.py` script is also provided, which can be triggered on-demand from the web dashboard. It targets a single user (`user_101`) with 15 rapid transactions to show how the system reacts to sudden spikes.

3.2 Fraud Detection Engine (Consumer)

The `consumer.py` script is the analytical core of the project. It belongs to the `fraud-detection-group` and continuously reads from the `transactions` topic.

For every message, it runs two checks:

1. **Amount Fraud:** A simple stateless check. If the transaction amount is strictly greater than 5000 EUR, it triggers an alert.
2. **Velocity Fraud:** A stateful check. A sliding window is implemented in Python using a dictionary. The consumer keeps track of the timestamps of the recent transactions for every user. If a user makes more than 3 transactions in a 10-second window, it is flagged as a velocity fraud.

When a fraud is found, the consumer performs three actions: it saves the full alert details into MongoDB, it produces a message into the `fraud-alerts` topic, and it sends a push notification to an administrator via Telegram.

Alert Lifecycle

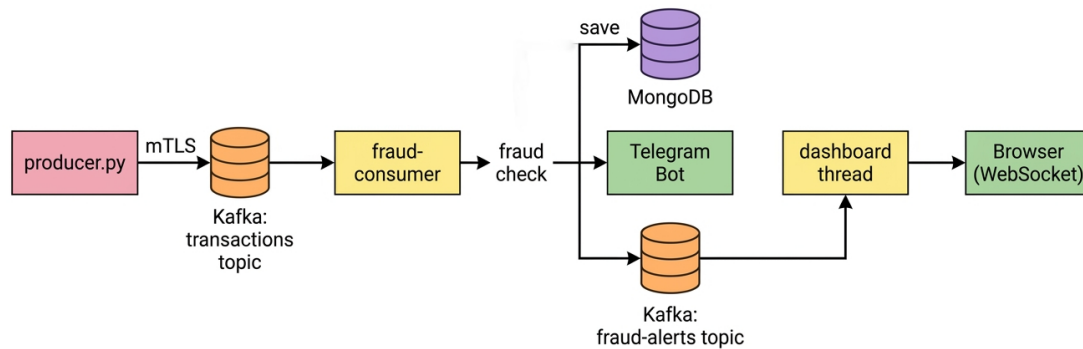


Figure 2: Alert lifecycle.

3.3 Real-Time Dashboard

To provide a visual representation of the incoming frauds, a lightweight web server is built using FastAPI. The server runs a background thread connected to Kafka (in its own **dashboard-group**), listening to the **fraud-alerts** topic.

When an alert arrives, the dashboard pushes it instantly to the web browser using WebSockets. The web page also has buttons to manually trigger the producers and test the system. Since it also connects to MongoDB, refreshing the page does not clear the past alerts.

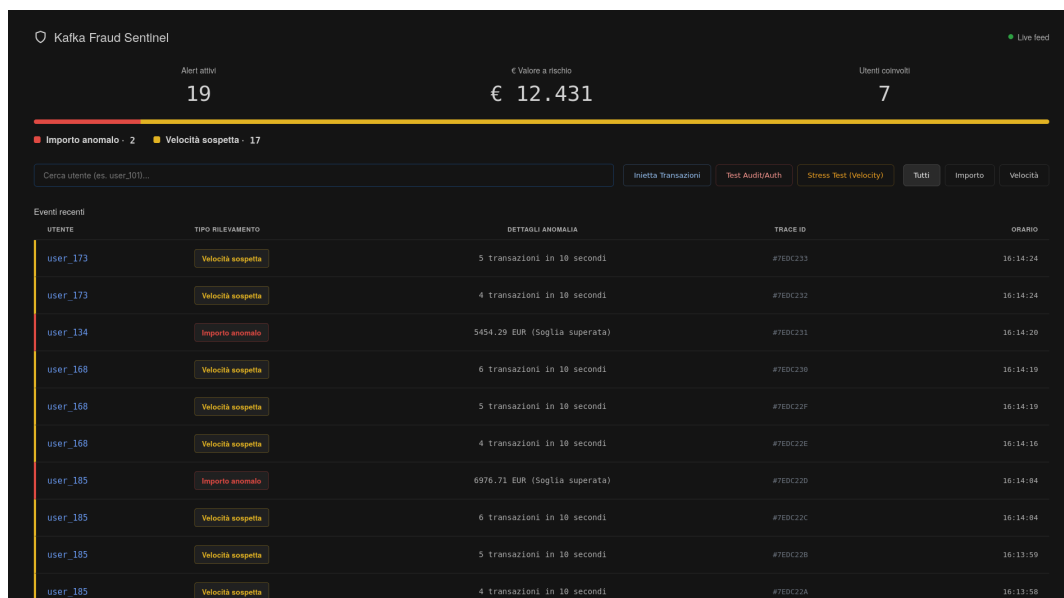


Figure 3: The web dashboard showing real-time alerts.

4 Security Implementation

Implementing security is a core requirement of the project. No microservice is allowed to connect to Kafka without proving its identity.

4.1 Mutual TLS (mTLS)

Instead of using simple passwords, mTLS is used. A bash script acts as a custom Certificate Authority (CA). The script generates certificates for the 3 Kafka brokers and for the 3 roles of the applications: `producer`, `consumer`, and `admin` (for the dashboard).

When a Python script connects to Kafka, it presents its certificate. If the certificate is missing or invalid, Kafka drops the connection immediately during the TLS handshake. This ensures all traffic is encrypted and only authorized scripts can connect.

4.2 Access Control Lists (ACLs)

After authenticating, a script must be authorized. The Kafka ACLs are configured so that every certificate has only the strict permissions it needs. For example:

- The producer can write to `transactions`, but cannot read from it.
- The fraud-consumer can read from `transactions` and write to `fraud-alerts`.
- The dashboard admin can only read from `fraud-alerts`.

An interesting issue with KRaft was encountered during development. Initially, the `ANONYMOUS` user was removed from the super-users list, but the brokers crashed at startup. It was discovered that the internal KRaft controllers communicate using the `ANONYMOUS` principal before the ACL engine is fully loaded. It had to be restored, but the risk is minimal because the internal ports are not exposed outside the Docker network.

5 Testing and Demos

Several scenarios are provided to demonstrate that the system works as expected under adverse conditions.

5.1 Broker Crash Survival

A bash script (`demo-broker-failure.sh`) violently stops one of the Kafka brokers (`kafka-2`) while the producer is running. Because of the configuration (`acks=all, min.insync.replicas=2`), the producer does not crash and no data is lost. The system simply continues working with the remaining two brokers. When `kafka-2` is restarted, it syncs back up automatically.

5.2 Security Tests

Two "attacker" scripts are included to test the security boundaries. One tries to connect in plaintext, and the other tries to connect with SSL but without the client certificate. In both cases, Kafka rejects the connection and the dashboard does not show any fake transactions.

5.3 Offset Recovery

A `recovery_demo.py` script is included to show how consumer offsets work. If a consumer crashes while reading, Kafka remembers exactly where it stopped. When the script is restarted, it resumes processing from the exact same point without duplicating or skipping messages.

5.4 Telegram Notifications

To ensure that system administrators are immediately informed of suspicious activities, a Telegram bot integration is implemented. If the bot credentials are provided in the environment variables, the fraud-consumer automatically dispatches a Markdown-formatted alert message to the designated chat. This provides a mobile-friendly monitoring solution alongside the web dashboard.



Figure 4: Telegram alerts delivered directly to the administrator’s phone.

6 Limitations and Future Improvements

This project serves as a solid prototype, but for a real production environment, a few architectural changes would be necessary:

- **Externalize State:** The velocity fraud counter is kept in a Python dictionary. If the `consumer.py` crashes, the memory is wiped and the current 10-second window is lost. This state should be moved to a Redis database to make it persistent across restarts.
- **Dynamic Rules:** Currently, the 5000 EUR limit is hardcoded. It would be better to fetch this threshold from a database or use a machine learning model to detect anomalies based on a user’s past behavior.
- **Exactly-Once Semantics:** If the consumer writes to MongoDB and crashes right before

sending the message to the **fraud-alerts** topic, the system might process that transaction twice when it restarts. Implementing exactly-once semantics would solve this edge case.

7 Conclusion

The development of "Kafka Fraud Sentinel" demonstrates exactly why standard databases or REST APIs are not suited for streaming events. Setting up the Kafka cluster with KRaft and writing the producer/consumer logic is straightforward, but configuring mTLS and ACLs correctly is challenging and highly instructive. Seeing the system survive a live broker crash without dropping a single transaction effectively proves the resilience of the architecture.